

Tugas 6: Praktikum Mandiri Support Vector Machine (SVM)

Rika Rahma - 0110222134 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: rika22134ti@student.nurulfikri.ac.id

Abstract. Pada praktikum ini dilakukan implementasi algoritma Support Vector Machine (SVM) untuk melakukan klasifikasi terhadap Dry Bean Dataset yang diambil dari Kaggle. Dataset ini berisi 13.611 data biji kacang yang terdiri dari 16 fitur numerik seperti Area, Perimeter, MajorAxisLength, MinorAxisLength, dan Compactness, serta satu kolom target yaitu Class yang berisi tujuh jenis kacang berbeda. Hasil evaluasi menunjukkan bahwa model SVM mampu mengklasifikasikan jenis kacang dengan akurasi sebesar 92,43%, dengan nilai precision, recall, dan f1-score rata-rata di atas 0,90, yang menandakan performa model sangat baik dan stabil pada setiap kelas. Visualisasi Confusion Matrix memperlihatkan sebagian besar prediksi berada pada diagonal utama, menunjukkan kesalahan klasifikasi yang sangat kecil. Sementara itu, visualisasi 2D antara fitur Area dan Perimeter memperlihatkan pembentukan kelompok (cluster) yang jelas antarjenis kacang, menegaskan bahwa fitur tersebut efektif dalam membedakan tiap kelas. Secara keseluruhan, algoritma SVM terbukti efisien dan akurat dalam melakukan multi-class classification pada Dry Bean Dataset.

1.1. Import Library

Kode:

1. Import Library

```
# Import Library
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Gambar 1. Import Library

Penjelasan:

Pada bagian ini, kita mengimpor berbagai library Python yang dibutuhkan untuk proses implementasi algoritma Support Vector Machine (SVM). Library yang digunakan antara lain:

- pandas → digunakan untuk membaca, memproses, dan menganalisis data dalam bentuk DataFrame.
- numpy → berfungsi untuk melakukan operasi numerik dan manipulasi array.

- seaborn dan matplotlib.pyplot → dipakai untuk visualisasi data, seperti membuat grafik distribusi kelas dan confusion matrix.
- train_test_split dari sklearn.model_selection → digunakan untuk membagi dataset menjadi dua bagian, yaitu data latih (training) dan data uji (testing).
- StandardScaler dan LabelEncoder dari sklearn.preprocessing → StandardScaler digunakan untuk menstandarkan skala fitur agar model bekerja lebih optimal, sedangkan LabelEncoder digunakan untuk mengubah label teks menjadi angka.
- SVC (Support Vector Classifier) dari sklearn.svm → merupakan algoritma utama Support Vector Machine yang digunakan untuk melakukan proses klasifikasi.
- accuracy_score, classification_report, dan confusion_matrix dari sklearn.metrics → digunakan untuk mengukur performa model dengan menghitung akurasi, precision, recall, F1-score, serta menampilkan confusion matrix.

1.2. Load Dataset

Kode:

```
2. Load Dataset

# Menghubungkan colab dengan Google Drive
from google.colab import drive
drive.mount('/content/gdrive')

Mounted at /content/gdrive

# Memanggil dataset lewat Gdrive
path = '/content/gdrive/MyDrive/Praktikum_ML/praktikum06/data/'

# Membaca file CSV menggunakan pandas
df = pd.read_csv(path + 'Dry_Bean_Dataset.csv')
df.head()
```

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRatio	Eccentricity	ConvexArea	EquiVlaminar	Extent	Solidity	roundness	Compactness	ShapeFactor1	ShapeFactor2	ShapeFactor3	ShapeFactor4	Class
0	28395	610.291	208.178117	173.888747	1.197191	0.549612	26715	190.141097	0.763923	0.980856	0.958027	0.913358	0.007332	0.003147	0.834222	0.998724	SEKER
1	28734	638.018	200.524796	182.734419	1.097356	0.411785	29172	191.272751	0.783968	0.984986	0.887034	0.953861	0.006979	0.003564	0.909851	0.998430	SEKER
2	29380	624.110	212.826130	175.931143	1.209713	0.562727	29690	193.410904	0.778113	0.989559	0.947849	0.908774	0.007244	0.003048	0.825871	0.999066	SEKER
3	30008	645.884	210.557999	182.516516	1.153638	0.498616	30724	195.467062	0.782681	0.976696	0.903936	0.928329	0.007017	0.003215	0.861794	0.994199	SEKER
4	30140	620.134	201.847882	190.279279	1.060798	0.333680	30417	195.896503	0.772098	0.990893	0.984877	0.970516	0.006697	0.003665	0.941900	0.999166	SEKER

Gambar 2. Load Dataset

Penjelasan:

- from google.colab import drive dan drive.mount('/content/gdrive') → digunakan untuk menghubungkan Google Colab dengan Google Drive agar file dataset bisa diakses.
- path = '/content/gdrive/MyDrive/Praktikum_ML/praktikum06/data/' → menentukan lokasi folder tempat file dataset disimpan di Google Drive.
- pd.read_csv(path + 'Dry_Bean_Dataset.csv') → membaca file CSV menggunakan library pandas dan menyimpannya ke dalam variabel df dalam bentuk DataFrame.
- df.head() → menampilkan 5 baris pertama dataset untuk memastikan data berhasil dimuat dan melihat struktur awal dataset.
- Dataset Dry Bean berisi fitur numerik seperti Area, Perimeter, MajorAxisLength, MinorAxisLength, AspectRatio, Eccentricity, ConvexArea, Extent, Solidity, Compactness, dan ShapeFactor1–4, sedangkan kolom Class adalah label target berisi jenis kacang (SEKER, BARBUNYA, DERMASON, dll).

Kode:

```
# Cek informasi kolom dan tipe data
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13611 entries, 0 to 13610
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Area                   13611 non-null  int64
1   Perimeter              13611 non-null  float64
2   MajorAxisLength        13611 non-null  float64
3   MinorAxisLength        13611 non-null  float64
4   AspectRatio            13611 non-null  float64
5   Eccentricity            13611 non-null  float64
6   ConvexArea              13611 non-null  int64
7   EquivDiameter           13611 non-null  float64
8   Extent                  13611 non-null  float64
9   Solidity                13611 non-null  float64
10  roundness               13611 non-null  float64
11  Compactness             13611 non-null  float64
12  ShapeFactor1            13611 non-null  float64
13  ShapeFactor2            13611 non-null  float64
14  ShapeFactor3            13611 non-null  float64
15  ShapeFactor4            13611 non-null  float64
16  Class                   13611 non-null  object
dtypes: float64(14), int64(2), object(1)
memory usage: 1.8+ MB

print("Jumlah Data Tiap Kelas:")
print(df['Class'].value_counts())

Jumlah Data Tiap Kelas:
Class
DERMASON    3546
SIRA        2636
SEKER       2827
HOROZ       1928
CALI        1630
BARBUNYA    1322
BOMBAY       522
Name: count, dtype: int64
```

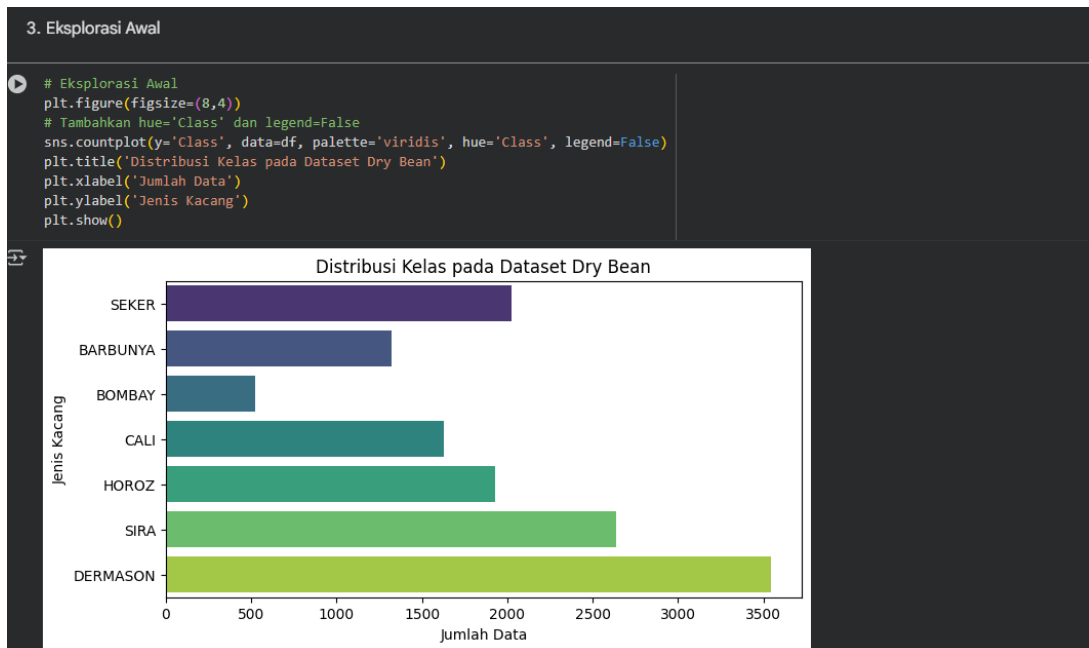
Gambar 3. Cek Informasi Data

Penjelasan:

- `df.info()` → digunakan untuk menampilkan informasi umum tentang dataset, seperti jumlah baris, jumlah kolom, nama kolom, tipe data, serta apakah ada data yang kosong (null).
- Hasilnya menunjukkan bahwa dataset memiliki 13.611 baris dan 17 kolom, di mana 16 kolom bertipe numerik (int64 atau float64) dan 1 kolom bertipe object yaitu Class (label target). Tidak ada data yang hilang karena semua kolom memiliki nilai non-null.
- Perintah `print(df['Class'].value_counts())` digunakan untuk menghitung jumlah data pada setiap kelas target.
- Hasilnya menunjukkan terdapat 7 jenis kacang (Class), yaitu DERMASON, SIRA, SEKER, HOROZ, CALI, BARBUNYA, dan BOMBAY.
- Jumlah data terbanyak terdapat pada kelas DERMASON (3546 data), sedangkan yang paling sedikit adalah BOMBAY (522 data).

1.3. Eksplorasi

Kode:



Gambar 4. Eksplorasi awal

Penjelasan:

- `feature_columns` berisi nama-nama kolom yang digunakan sebagai fitur untuk model (panjang & lebar sepal dan petal).
- Kode `plt.figure(figsize=(8,4))` digunakan untuk menentukan ukuran tampilan grafik agar hasil visualisasi lebih proporsional dan mudah dibaca.
- `sns.countplot(y='Class', data=df, palette='viridis', hue='Class', legend=False)` berfungsi untuk menampilkan grafik distribusi kelas (Class) pada dataset menggunakan Seaborn. Grafik ini menunjukkan jumlah data dari masing-masing jenis kacang.
- `plt.title('Distribusi Kelas pada Dataset Dry Bean')` memberikan judul pada grafik, sedangkan `plt.xlabel('Jumlah Data')` dan `plt.ylabel('Jenis Kacang')` digunakan untuk memberi label pada sumbu X dan Y.
- `plt.show()` digunakan untuk menampilkan grafik hasil visualisasi.
- Dari grafik terlihat bahwa jumlah data pada setiap kelas tidak seimbang. Kelas DERMAISON memiliki jumlah data terbanyak (lebih dari 3.500), sedangkan BOMBAY memiliki data paling sedikit (sekitar 500).

1.4. Pemisahan Fitur dan Label

Kode:

```
4. Pemisahan Fitur dan Label

# Memisahkan Fitur dan Label
X = df.drop('Class', axis=1)
y = df['Class']

# Label encoding karena target berupa teks
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Gambar 5. Pemisahan Fitur dan Label

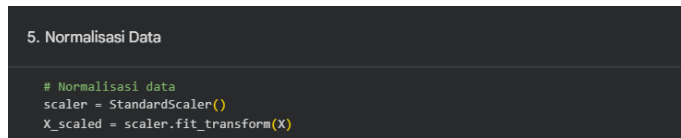
Penjelasan:

- `x` adalah fitur (input) dan `y` adalah target (label).

- `X = df.drop('Class', axis=1)` dan `y = df['Class']` → digunakan untuk memisahkan fitur dan label. Semua kolom kecuali Class disimpan sebagai variabel X (fitur), sedangkan kolom Class disimpan sebagai variabel y (target/label).
- `LabelEncoder()` digunakan karena target masih berbentuk teks (nama jenis kacang). Perintah `y_encoded = le.fit_transform(y)` mengubah label teks menjadi angka agar dapat diproses oleh model machine learning.

1.5. Normalisasi Data

Kode:



```
5. Normalisasi Data

# Normalisasi data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

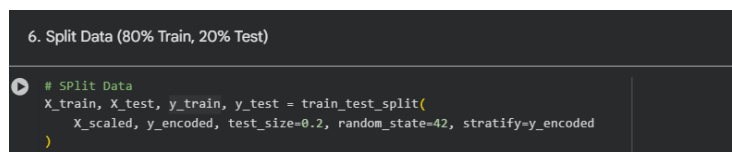
Gambar 6. Normalisasi Data

Penjelasan:

- `StandardScaler()` digunakan untuk menormalkan skala data fitur agar setiap kolom memiliki rentang nilai yang sebanding. Proses ini penting karena algoritma SVM sangat sensitif terhadap skala fitur. Hasil normalisasi disimpan ke dalam variabel `X_scaled`.

1.6. Split Data

Kode:



```
6. Split Data (80% Train, 20% Test)

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
```

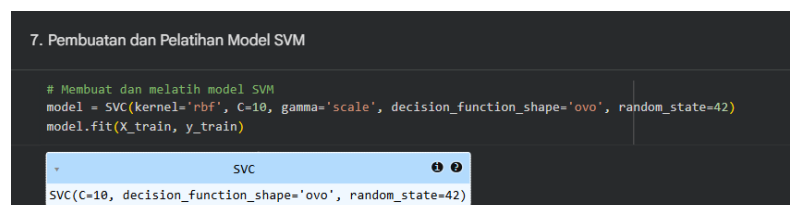
Gambar 7. Split Data

Penjelasan:

- `train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded)` digunakan untuk membagi dataset menjadi 80% data latih dan 20% data uji. Parameter `stratify` memastikan proporsi setiap kelas tetap seimbang antara data latih dan uji.

1.7. Pembuatan dan Pelatihan Model

Kode:



```
7. Pembuatan dan Pelatihan Model SVM

# Membuat dan melatih model SVM
model = SVC(kernel='rbf', C=10, gamma='scale', decision_function_shape='ovo', random_state=42)
model.fit(X_train, y_train)
```

Gambar 8. Pembuatan dan Pelatihan Model

Penjelasan:

- `model = SVC(kernel='rbf', C=10, gamma='scale', decision_function_shape='ovo', random_state=42)` digunakan untuk membuat model Support Vector Machine (SVM) dengan kernel RBF. Parameter `C=10` mengontrol tingkat regularisasi, sedangkan `gamma='scale'` menyesuaikan pengaruh satu data terhadap batas keputusan model.

- `decision_function_shape='ovo'` (one-vs-one) berarti model akan membuat beberapa classifier untuk menangani kasus multi-class.
- `model.fit(X_train, y_train)` digunakan untuk melatih model SVM menggunakan data latih agar model dapat mengenali pola dari setiap kelas kacang berdasarkan fitur yang tersedia.

1.8. Evaluasi Model

Kode:

```
8. Evaluasi Model

y_pred = model.predict(X_test)

akurasi = accuracy_score(y_test, y_pred)
print("\nAkurasi Model: {:.2f}%".format(akurasi * 100))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))

Akurasi Model: 92.43%

Classification Report:
      precision    recall  f1-score   support

 BARBUNYA      0.93      0.91      0.92       265
  BOMBAY      1.00      1.00      1.00       104
    CALI      0.94      0.94      0.94       326
 DERMASON      0.91      0.92      0.92       709
  HOROZ      0.96      0.96      0.96       386
    SEKER      0.94      0.95      0.95       406
    SIRA      0.87      0.86      0.87       527

 accuracy      0.94      0.94      0.92      2723
  macro avg      0.94      0.94      0.94      2723
 weighted avg      0.92      0.92      0.92      2723
```

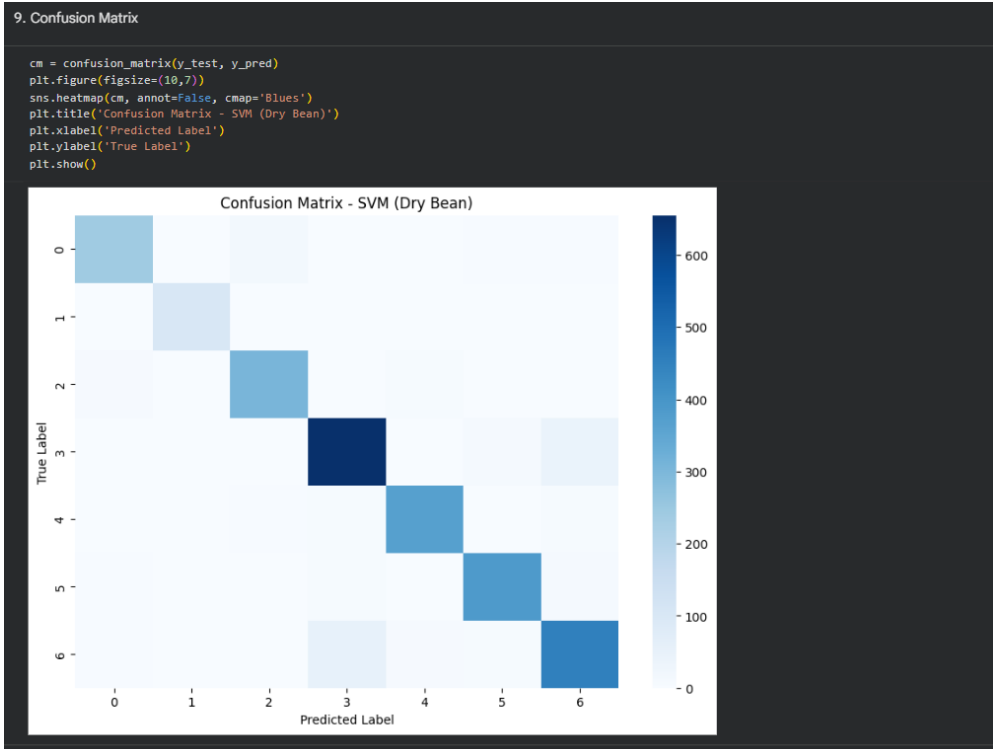
Gambar 9. Evaluasi Model

Penjelasan:

- `y_pred = model.predict(X_test)` → digunakan untuk memprediksi data uji berdasarkan model SVM yang telah dilatih.
- `accuracy_score(y_test, y_pred)` → menghitung tingkat akurasi model, yaitu persentase prediksi yang benar dari seluruh data uji.
- `print("Akurasi Model: {:.2f}%".format(akurasi * 100))` → menampilkan hasil akurasi model dalam bentuk persentase.
- `classification_report(y_test, y_pred, target_names=le.classes_)` → menampilkan laporan evaluasi berupa precision, recall, f1-score, dan support untuk tiap kelas.
- **Output:** Akurasi model sebesar 92,43%, menunjukkan model SVM mampu mengklasifikasikan jenis kacang dengan sangat baik. Nilai precision, recall, dan f1-score yang rata-rata di atas 0.90 menandakan kinerja model stabil dan seimbang pada setiap kelas.

1.9. Confusion Matrix

Kode:



Gambar 10. Confusion Matrix

Penjelasan:

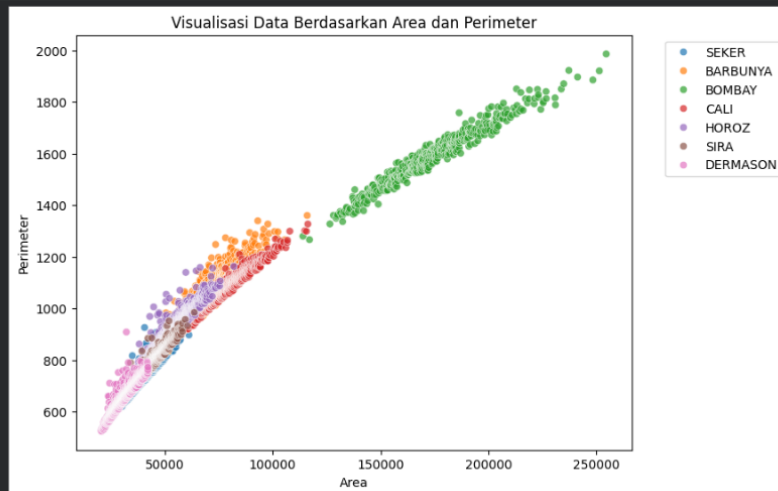
- `cm = confusion_matrix(y_test, y_pred)` → digunakan untuk membuat confusion matrix, yaitu tabel yang membandingkan antara label sebenarnya (true label) dan hasil prediksi model (predicted label).
- `plt.figure(figsize=(10,7))` → mengatur ukuran tampilan grafik agar hasil visualisasi lebih jelas dan proporsional.
- `sns.heatmap(cm, annot=False, cmap='Blues')` → menampilkan confusion matrix dalam bentuk heatmap berwarna biru, di mana semakin gelap warnanya menunjukkan jumlah prediksi benar yang lebih banyak.
- `plt.title('Confusion Matrix - SVM (Dry Bean)')`, `plt.xlabel('Predicted Label')`, dan `plt.ylabel('True Label')` → digunakan untuk memberikan judul serta label pada sumbu x dan y agar grafik mudah dipahami.
- **Output:** Grafik menunjukkan bahwa sebagian besar titik berwarna gelap berada pada diagonal utama, yang berarti model SVM berhasil memprediksi sebagian besar kelas dengan benar.

1.10. Visualisasi Fitur (2D)

Kode:

10. Visualisasi Fitur (2D)

```
plt.figure(figsize=(8,6))
sns.scatterplot(x=df['Area'], y=df['Perimeter'], hue=df['Class'], palette='tab10', alpha=0.7)
plt.title('Visualisasi Data Berdasarkan Area dan Perimeter')
plt.xlabel('Area')
plt.ylabel('Perimeter')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Gambar 11. Visualisasi Fitur (2D)

Penjelasan:

- `le` adalah `LabelEncoder` yang digunakan untuk mengubah label teks menjadi angka saat pelatihan.
- `plt.figure(figsize=(8,6))` → digunakan untuk mengatur ukuran kanvas gambar agar visualisasi lebih proporsional dan mudah dibaca.
- `sns.scatterplot(x=df['Area'], y=df['Perimeter'], hue=df['Class'], palette='tab10', alpha=0.7)` → membuat scatter plot 2D antara fitur Area (sumbu X) dan Perimeter (sumbu Y).
- Parameter `hue=df['Class']` memberi warna berbeda untuk setiap jenis kacang.
- `palette='tab10'` menentukan kombinasi warna.
- `alpha=0.7` membuat titik sedikit transparan agar data yang saling menumpuk tetap terlihat.
- `plt.title()`, `plt.xlabel()`, dan `plt.ylabel()` → menambahkan judul grafik dan label pada sumbu X serta Y agar visualisasi informatif.
- `plt.legend()` → menampilkan legenda di sisi kanan untuk menunjukkan warna yang mewakili masing-masing kelas kacang.
- **Output:** Grafik menunjukkan bahwa setiap jenis kacang membentuk kelompok (cluster) yang relatif terpisah berdasarkan nilai Area dan Perimeter. Hal ini menandakan bahwa kedua fitur tersebut memiliki kemampuan yang baik dalam membedakan jenis kacang, sehingga sangat relevan digunakan dalam proses klasifikasi oleh model SVM.

2. Kesimpulan Implementasi

Berdasarkan hasil implementasi algoritma Support Vector Machine (SVM) pada *Dry Bean Dataset*, diperoleh akurasi sebesar 92,43% dengan nilai *precision*, *recall*, dan *f1-score* yang

tinggi pada setiap kelas. Hal ini menunjukkan bahwa model SVM dengan kernel RBF mampu mengklasifikasikan tujuh jenis kacang secara efektif. Visualisasi data dan *confusion matrix* juga memperlihatkan tingkat kesalahan yang rendah. Dengan demikian, SVM terbukti menjadi algoritma yang andal dalam menyelesaikan permasalahan multi-class classification pada data berkarakteristik numerik seperti *Dry Bean Dataset*.

3. Link Github

Praktikum 06:

<https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum06/notebook/praktikum06.ipynb>

Praktikum Mandiri:

https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum06/notebook/praktikum_mandiri06.ipynb

Dataset Kaggle (Dry Bean Dataset Classification):

<https://www.kaggle.com/datasets/nimapourmoradi/dry-bean-dataset-classification>